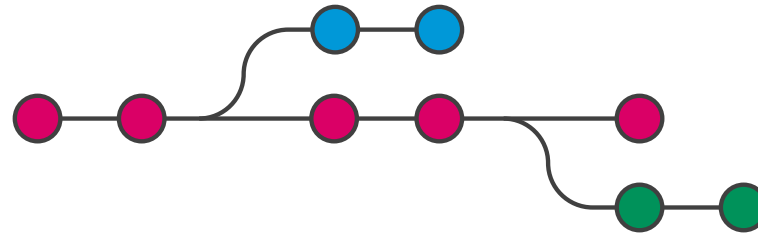




System Design

Version Control

Systems Engineering program



Silvan Zahno silvan.zahno@hevs.ch

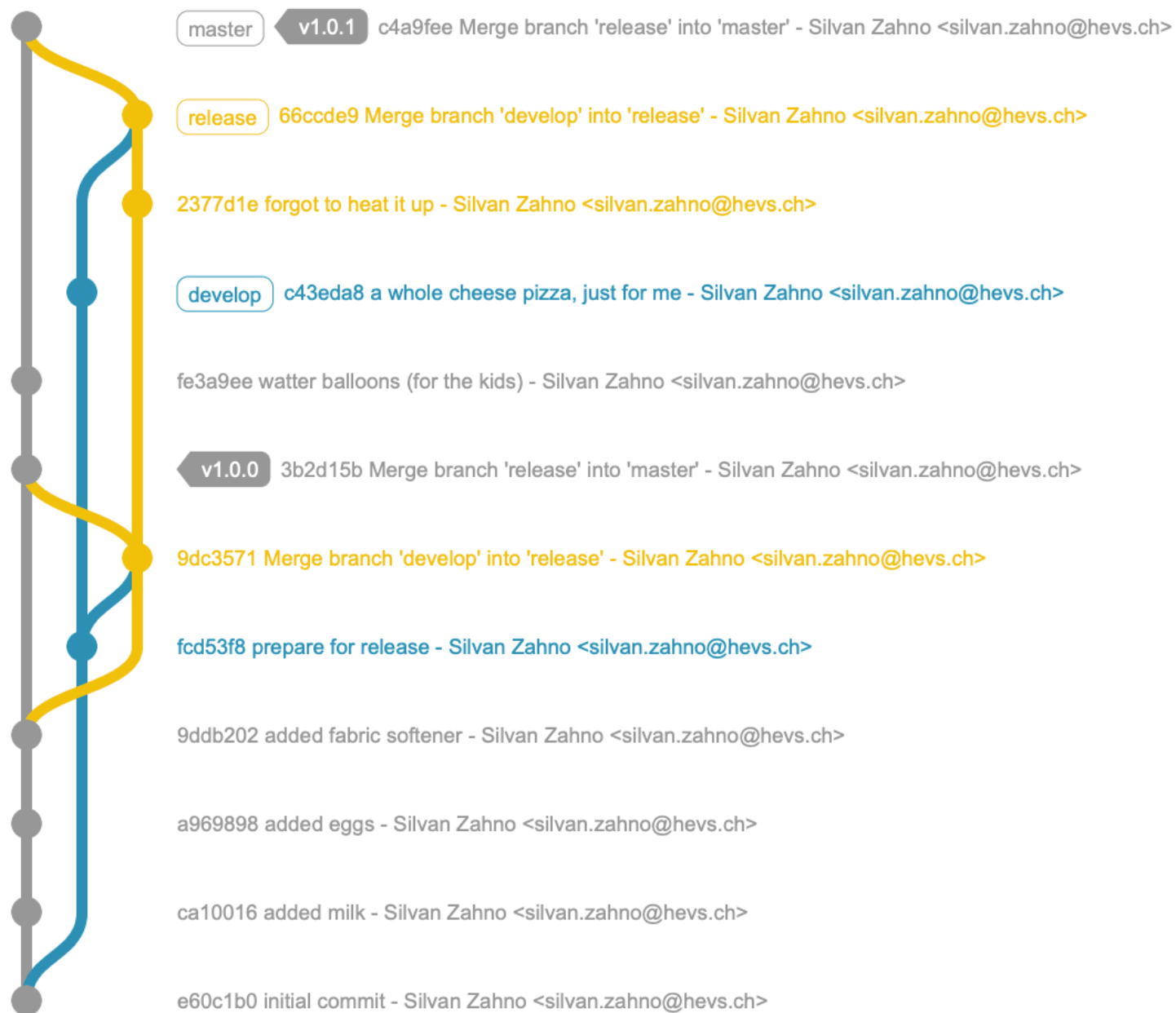
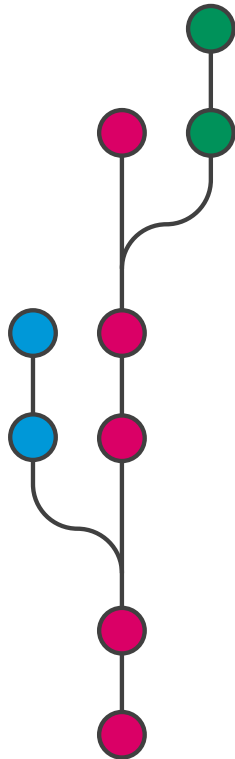
Your current system



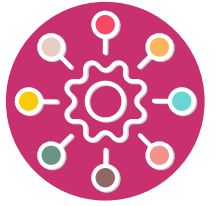
```
.
├── important-file copy.txt
├── important-file v1.txt
├── important-file v2.1 backup.txt
├── important-file v2.1 backup.txt.old
├── important-file v2.1.txt
├── important-file v2.txt
└── important-file.txt
```

1 directory, 7 files

Version control



Possible Tools



Git (git)



Subversion (svn)

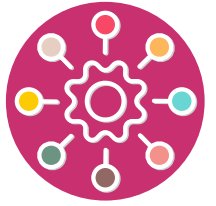
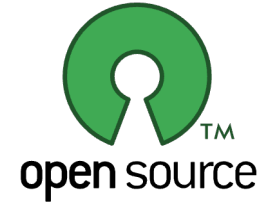


Mercurial (hg)

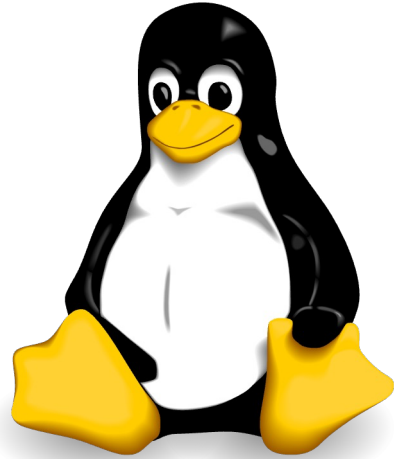


mercurial

Linus Torvalds



Linux (1991)



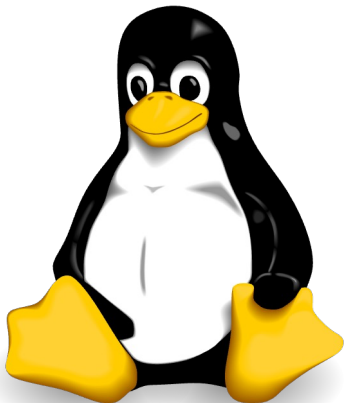
Git (2005)



Why Linux needs git



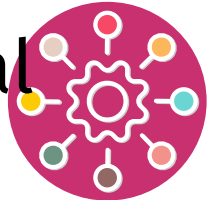
Linux has become the largest collaborative development project in the history of computing over the last 30 years.



- 600 active Linux distribution
- 85% of all Smartphones
- 500 top supercomputers
- >27.8 million lines of code
- > 12'000 contributors
- > 1 million commits

<https://truelist.co/blog/linux-statistics/>

Why you need git in Power&Control or Design&Material

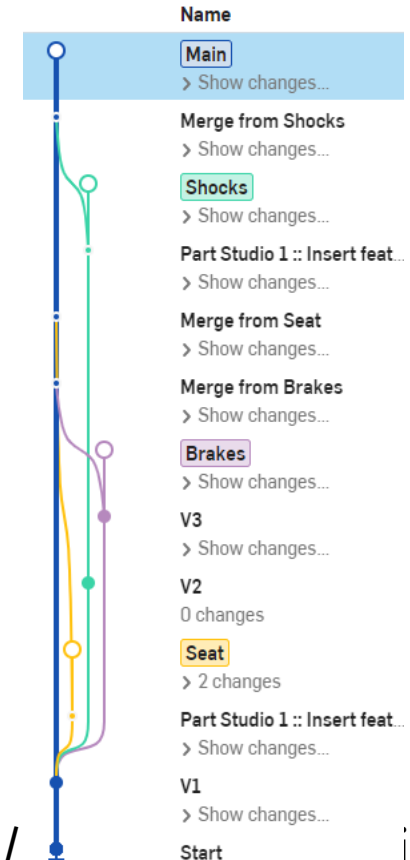


Versioning

- Tracking Changes
- Collaboration
- Rollback Capabilities
- Documentation
- Deployment



AUTODESK
Vault



<https://truelist.co/>

Git Platforms



Gitlab

<https://gitlab.com>

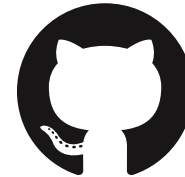
<https://gitlab.hevs.ch>



GitLab

Github

<https://github.com>



GitHub

Bitbucket

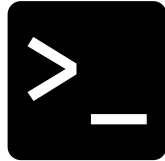
<https://bitbucket.com>



Bitbucket

Git Tools

Commandline



Sublime Merge



Git Cola



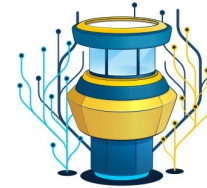
Git Kraken



Fork



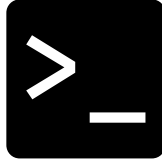
Tower



SmartGit



Git Commandline



git status

git diff

git add <file>

git diff --staged

git reset <file>

git commit -m "<commit message>"

git fetch <remote>

git merge <remote> <branch>

git push <remote> <branch>

git pull

```
zas@zas@zac ~ - zsh - 99x52
Last login: Tue Mar  8 09:26:26 on ttys004
zas@zac ~ (base)
$ git --help
usage: git [--version] [--help] [-C <path>] [-c <name>=<value>]
      [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
      [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
      [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
      [--super-prefix=<path>] [--config-env=<name>=<envvar>]
      <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
  bisect     Use binary search to find the commit that introduced a bug
  diff       Show changes between commits, commit and working tree, etc
  grep       Print lines matching a pattern
  log        Show commit logs
  show       Show various types of objects
  status     Show the working tree status

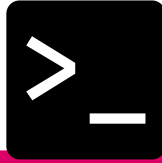
grow, mark and tweak your common history
  branch     List, create, or delete branches
  commit     Record changes to the repository
  merge      Join two or more development histories together
  rebase     Reapply commits on top of another base tip
  reset      Reset current HEAD to the specified state
  switch     Switch branches
  tag        Create, list, delete or verify a tag object signed with GPG

collaborate (see also: git help workflows)
  fetch      Download objects and refs from another repository
  pull       Fetch from and integrate with another repository or a local branch
  push       Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.

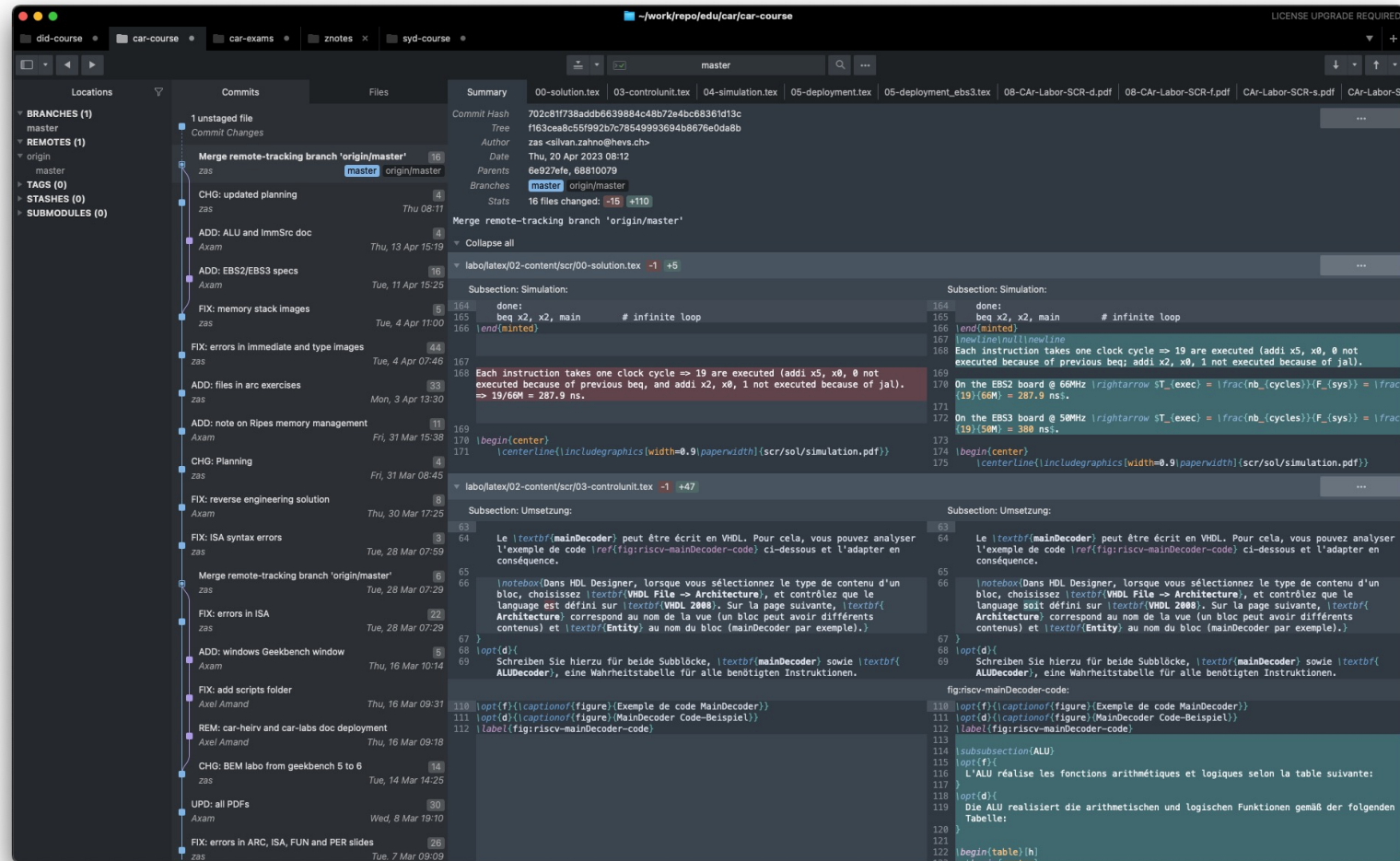
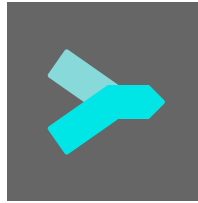
zas@zac ~ (base)
$
```

git Commands

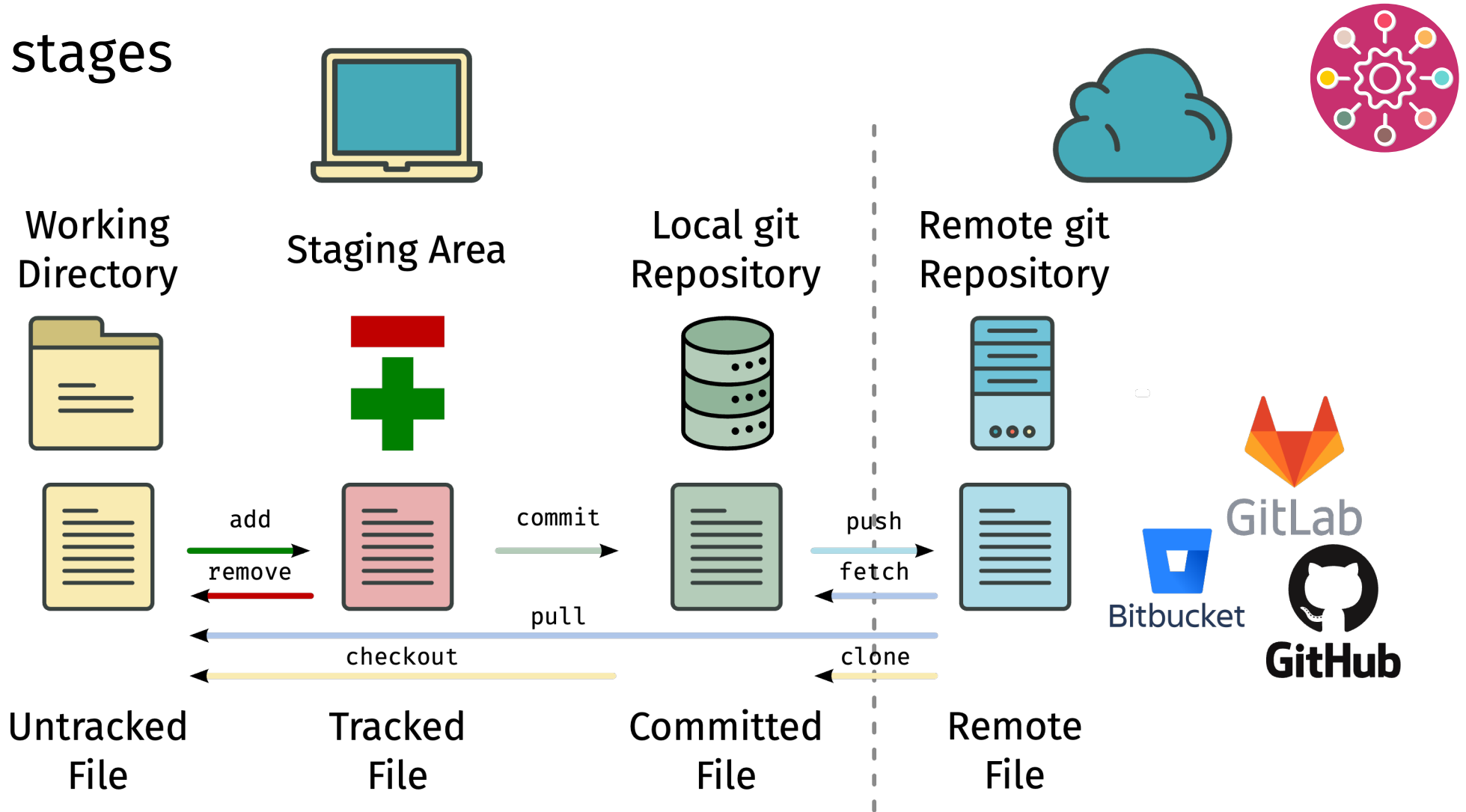


Command	Description	Command	Description
Start a working area		Grow, mark and tweak your common history	
clone	Clone a repository into a new directory	branch	List, create, or delete branches
init	Create an empty Git repository or reinitialize an existing one	checkout	Switch branches or restore working tree files
Work on the current change		commit	Record changes to the repo
add	Add file contents to the index	diff	Show changes between commits, commit and working tree, etc
mv	Move or rename a file, a directory, or a symlink	merge	Join two or more development histories together
reset	Reset current HEAD to the specified state	rebase	Reapply commits on top of another base tip
rm	Remove files from the working tree and from the index	tag	Create, list, delete or verify a tag object
Examine the history and state		Collaborate	
log	Show commit logs	fetch	Download objects and refs from another repo
show	Show various types of objects	pull	Fetch from and integrate with another repo or a local branch
status	Show the working tree status	push	Update remote refs along with associated objects

Sublime Merge

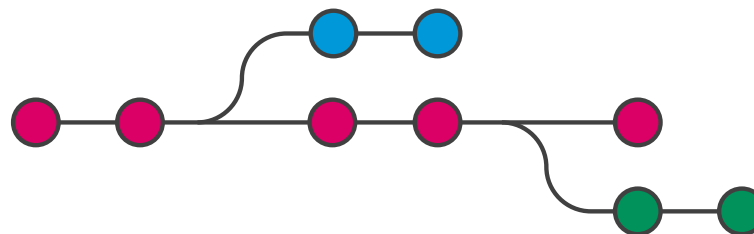


Git stages





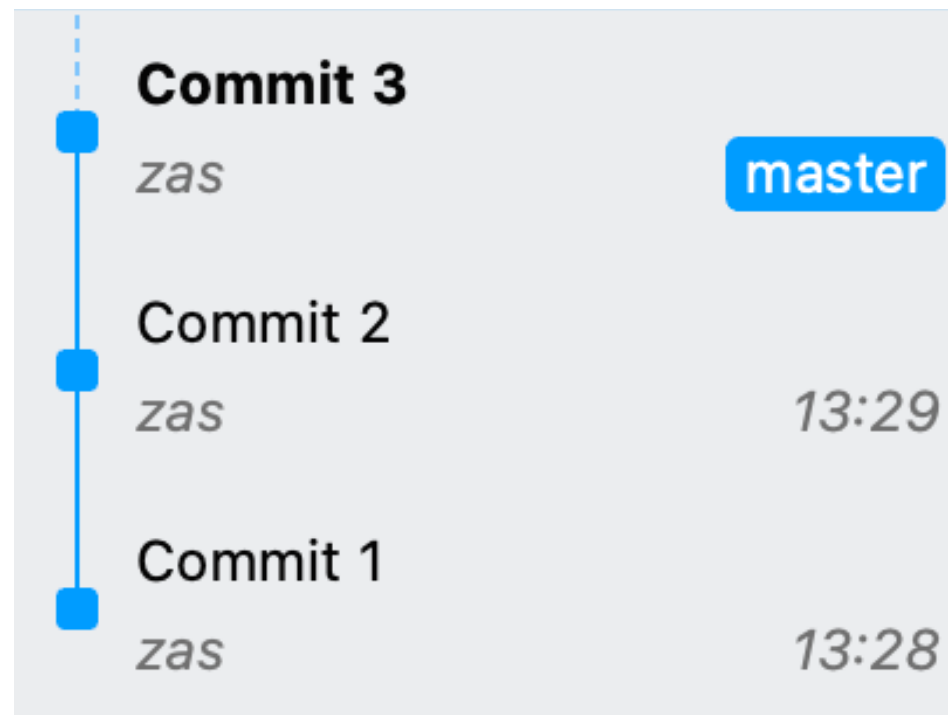
Git Branch and Merge Example



Branch and Merge

Initial repo state

Every repo has either a **main** or **master** branch as default branch



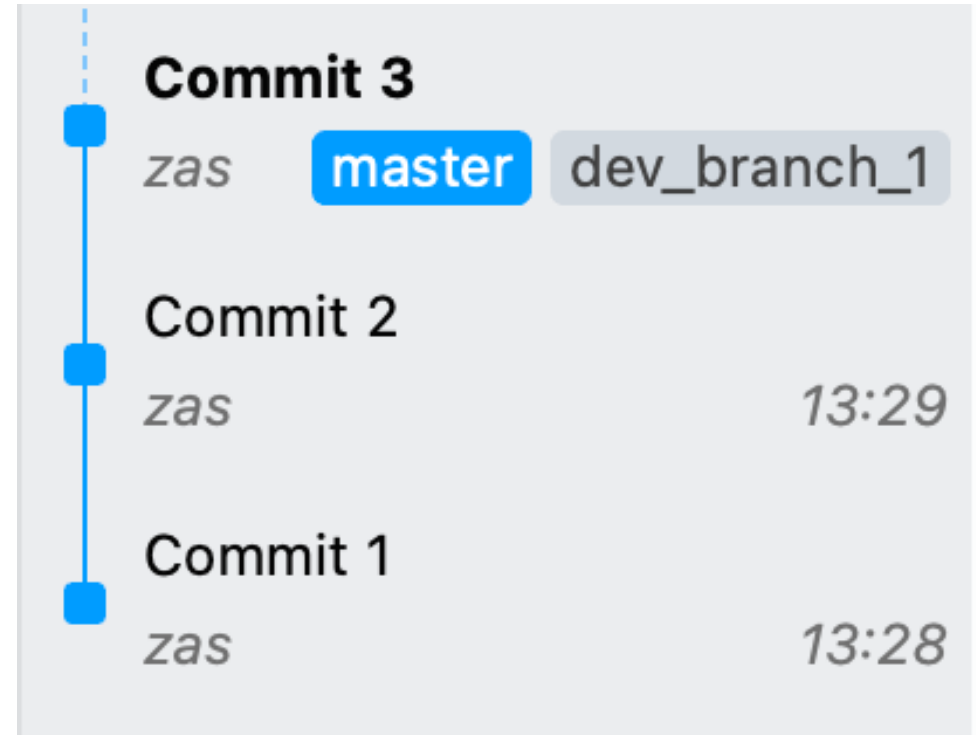
Branch and Merge

Create branch dev_branch_1



Create branch dev_branch_1

```
$ git branch dev_branch_1
```



Branch and Merge

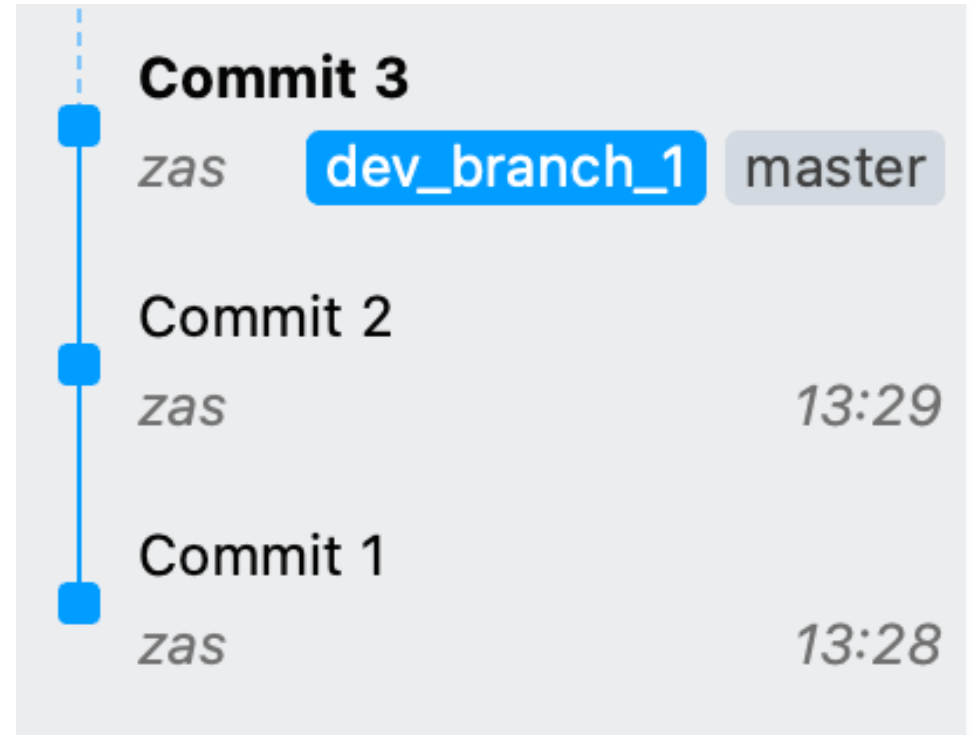
Checkout branch dev_branch_1

Check on which branch we are on

```
$ git branch
```

Checkout branch dev_branch_1

```
$ git checkout dev_branch_1
```



Branch and Merge

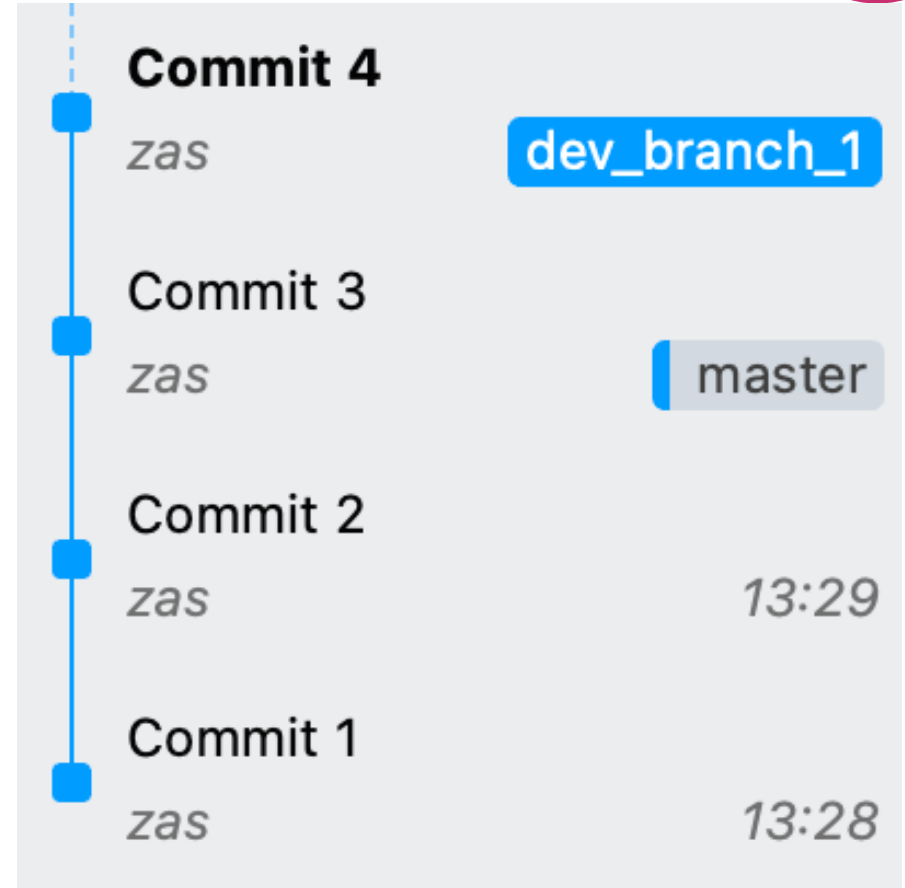
Commit on dev_branch_1

Stage new file

```
$ git add file.md
```

Commit stages files

```
$ git commit -m "Commit 4"
```



Branch and Merge

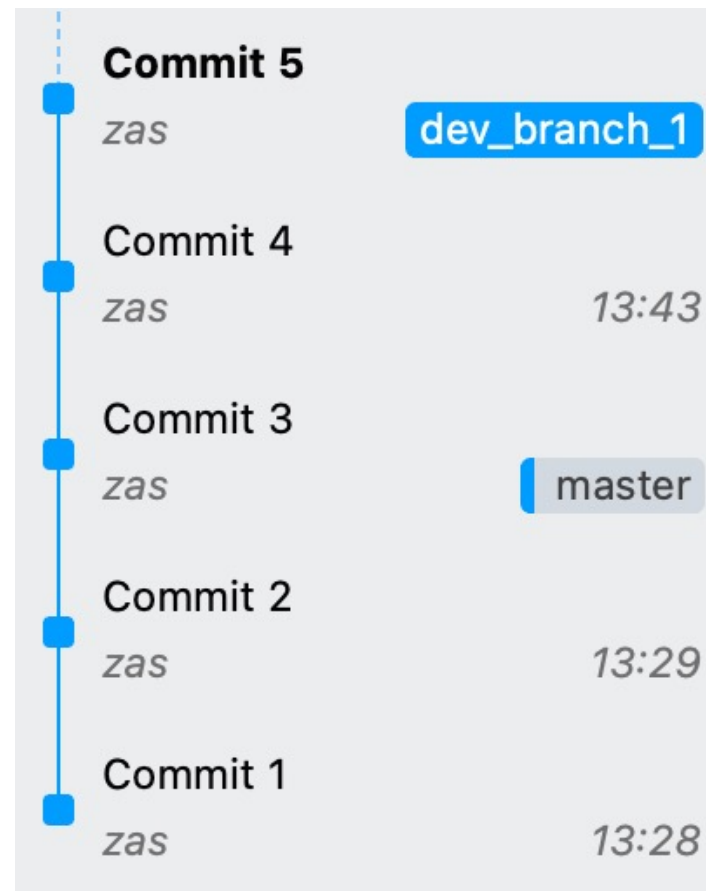
Commit on dev_branch_1

Stage new file

```
$ git add file.md
```

Commit stages files

```
$ git commit -m "Commit 5"
```



Branch and Merge

Commit on master branch

Checkout master branch

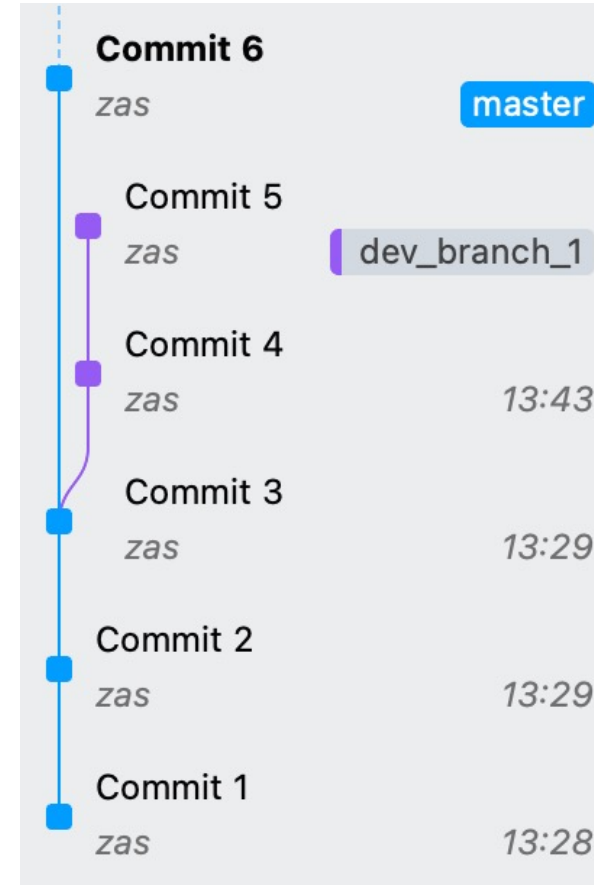
```
$ git checkout master
```

Stage new file

```
$ git add file.md
```

Commit stages files

```
$ git commit -m "Commit 6"
```



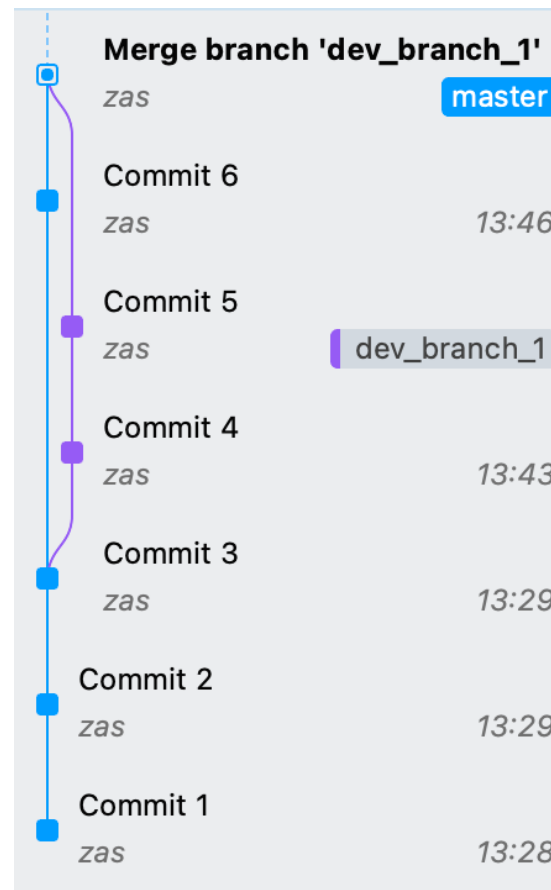
Branch and Merge

Three way merge master and dev_branch_1



Merge dev_branch_1 into master

```
$ git merge dev_branch_1
```



Demo



car-course

master

Locations

- BRANCHES (1)
- master
- remotes (1)
- origin
- master
- TAGS (0)
- STASHES (0)
- SUBMODULES (0)

Commits

- 1 unstaged file
- Commit Changes
- Merge remote-tracking branch 'origin/master'
- CHG: updated planning
- ADD: ALU and ImmSrc doc
- ADD: EBS2/EBS3 specs
- FIX: memory stack images
- FIX: errors in immediate and type images
- ADD: files in arc exercises
- ADD: note on Ripes memory management
- CHG: Planning
- FIX: reverse engineering solution
- FIX: ISA syntax errors
- Merge remote-tracking branch 'origin/master'
- FIX: errors in ISA
- ADD: windows Geekbench window
- FIX: add scripts folder
- REM: car-heiv and car-labs doc deployment
- CHG: BEM labo from geekbench 5 to 6
- UPD: all PDFs
- FIX: errors in ARC, ISA, FUN and PER slides

Files

- 00-solution.tex
- 03-controlunit.tex
- 04-simulation.tex
- 05-deployment.tex
- 05-deployment_ebs3.tex
- 08-CAR-Labor-SCR-d.pdf
- 08-CAR-Labor-SCR-f.pdf
- CAR-Labor-SCR-s.pdf
- CAR-Labor-SCR-t.pdf

Summary

Commit Hash: 702e81f738addb639884c48b72e4bc68361d13c

Author: zas <silvan.zahno@hevs.ch>

Date: Thu, 20 Apr 2023 08:12

Parents: 6e927efe, 68810079

Branches: master origin/master

Stats: 16 files changed: -15 +110

Merge remote-tracking branch 'origin/master'

Subsection: Simulation:

done: beq x2, x2, main # infinite loop

Each instruction takes one clock cycle => 19 are executed (addi x5, x0, 0 not executed because of previous beq; addi x2, x0, 1 not executed because of jal).

On the EBS2 board @ 66MHz $\rightarrow \frac{19}{66} = 287.9 \text{ ns}$.

On the EBS3 board @ 50MHz $\rightarrow \frac{19}{50} = 380 \text{ ns}$.

Subsection: Umsetzung:

Le `\texttt{mainDecoder}` peut être écrit en VHDL. Pour cela, vous pouvez analyser l'exemple de code `\ref{fig:riscv-mainDecoder-code}` ci-dessous et l'adapter en conséquence.

Dans HDL Designer, lorsque vous sélectionnez le type de contenu d'un bloc, choisissez `\texttt{VHDL File}` -> Architecture, et contrôlez que le langage est défini sur `\texttt{VHDL 2008}`. Sur la page suivante, `\texttt{Architecture}` correspond au nom de la vue (un bloc peut avoir différents contenus) et `\texttt{Entity}` au nom du bloc (`mainDecoder` par exemple).

Schreiben Sie hierzu für beide Subblöcke, `\texttt{mainDecoder}` sowie `\texttt{ALUDecoder}`, eine Wahrheitstabelle für alle benötigten Instruktionen.

figriscv-mainDecoder-code:

Example de code MainDecoder

MainDecoder Code-Beispiel

subsubsection{ALU}

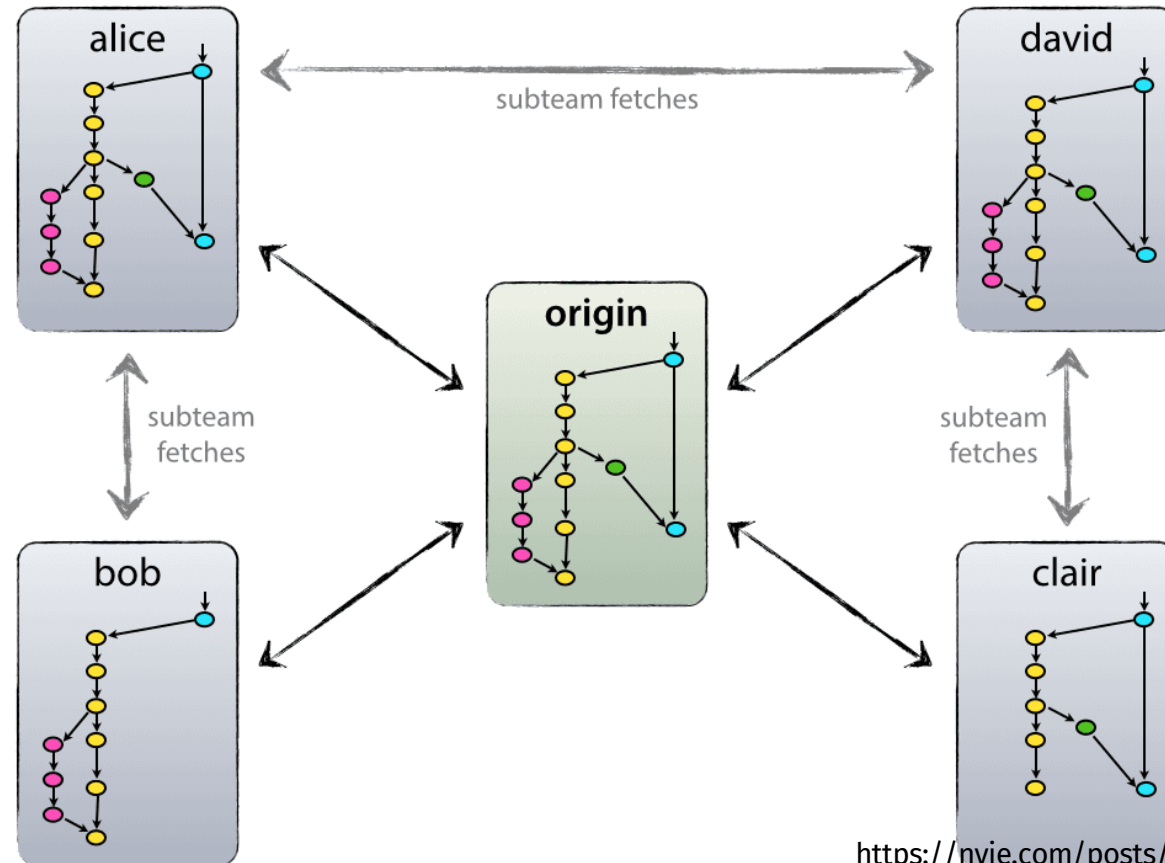
L'ALU réalise les fonctions arithmétiques et logiques selon la table suivante:

Die ALU realisiert die arithmetischen und logischen Funktionen gemäß der folgenden Tabelle:



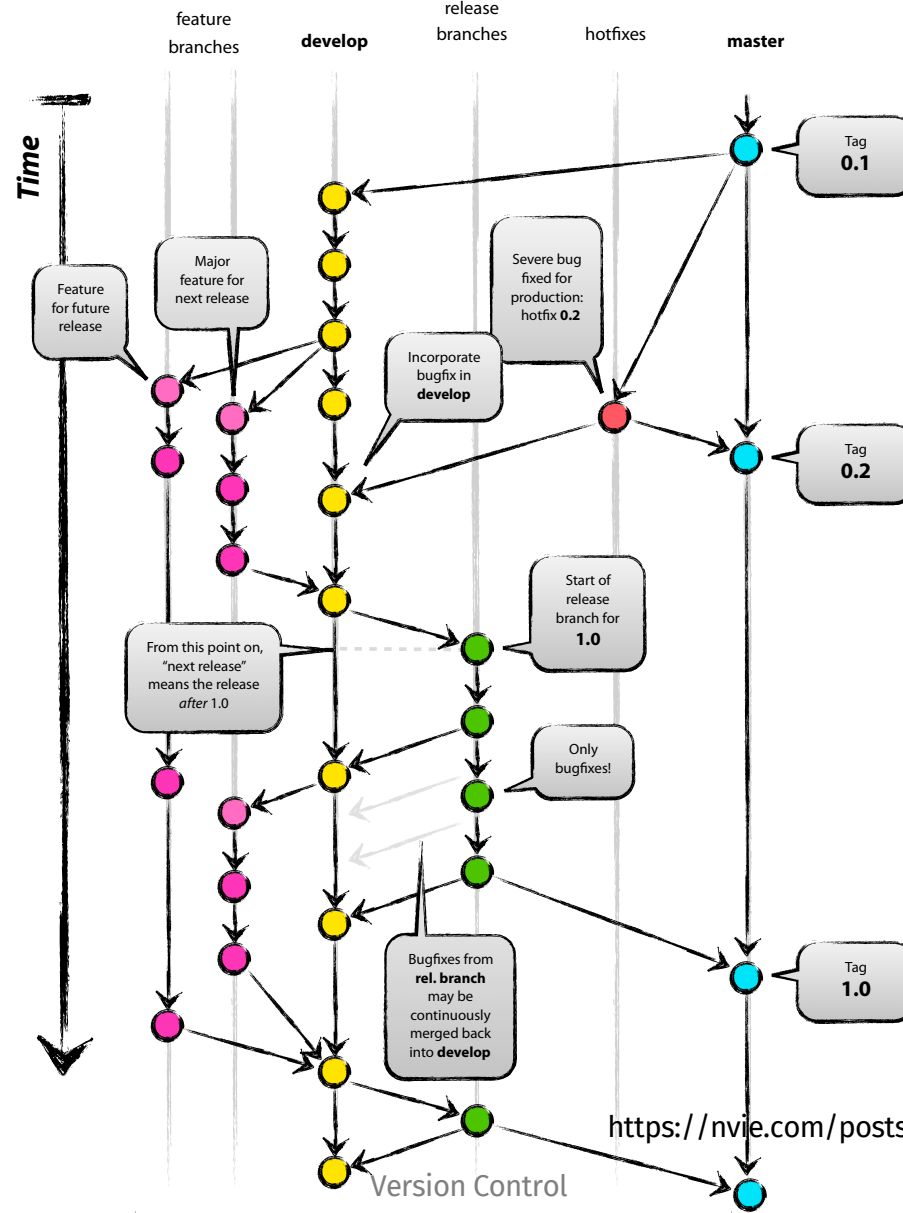
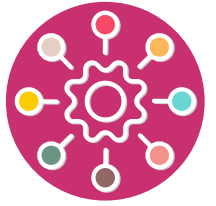
Gitflow

Gitflow Collaboration

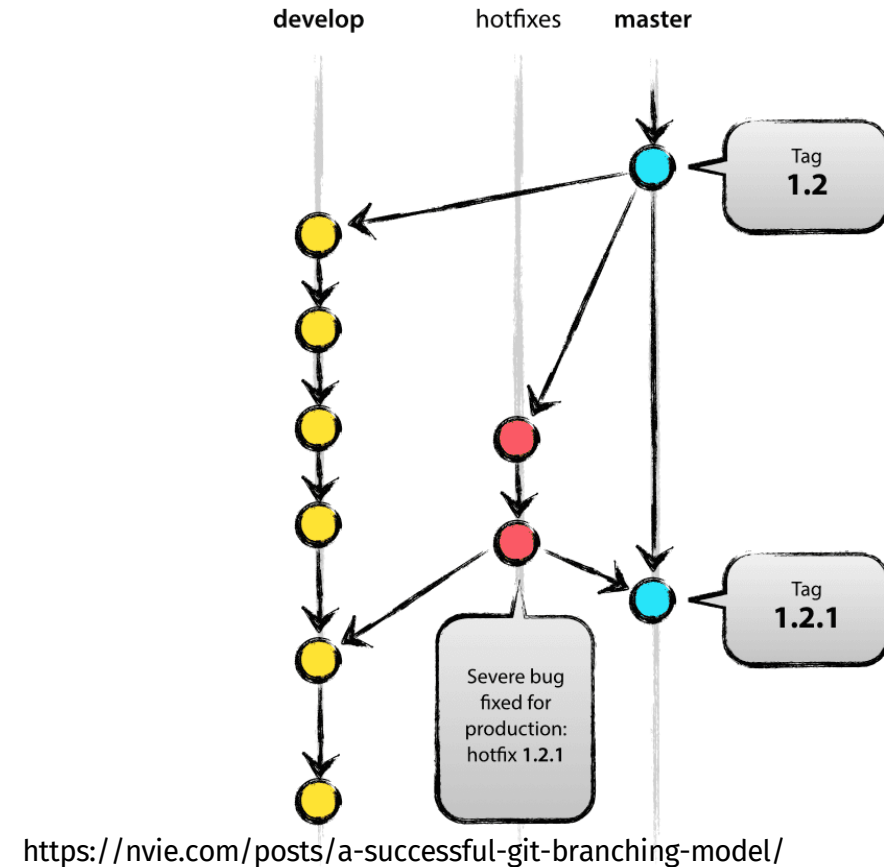
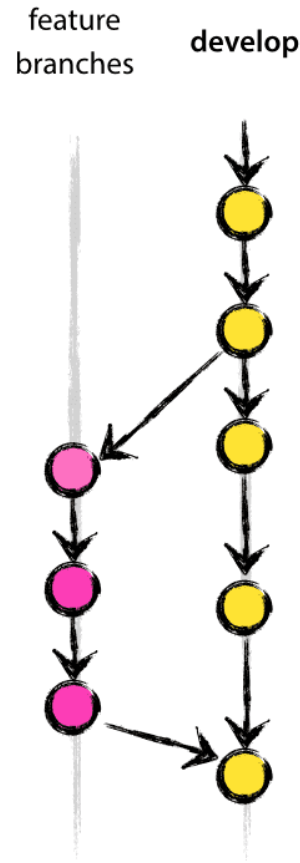
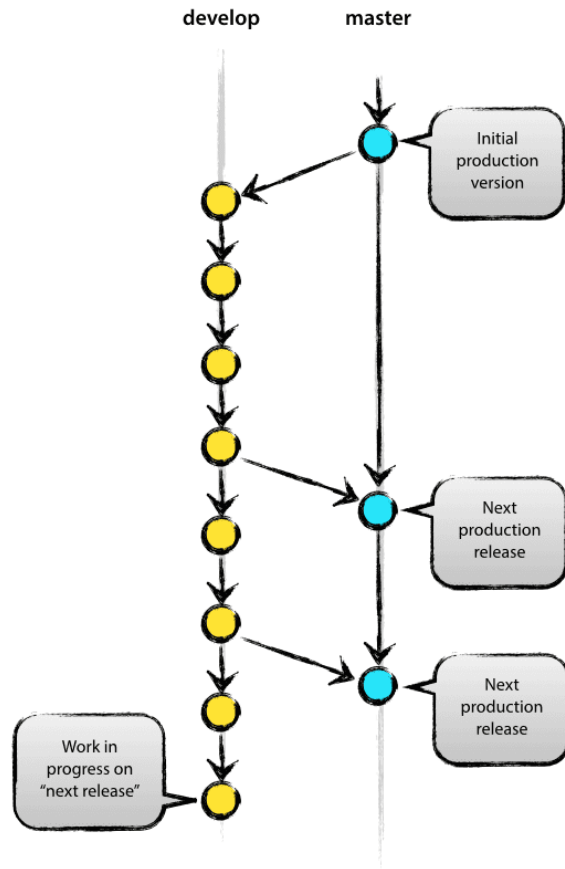


<https://nvie.com/posts/a-successful-git-branching-model/>

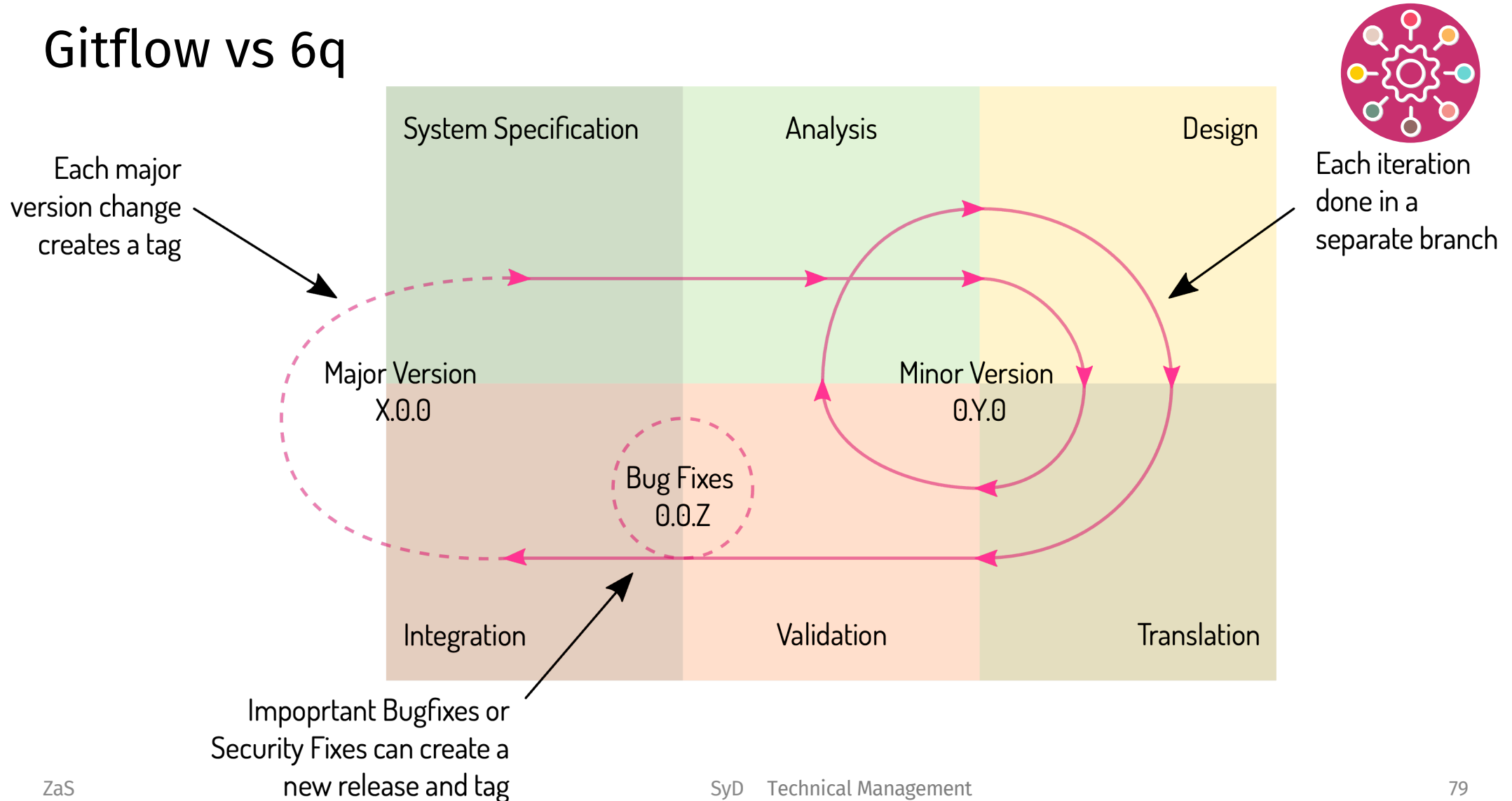
Gitflow



Gitflow Branching



Gitflow vs 6q





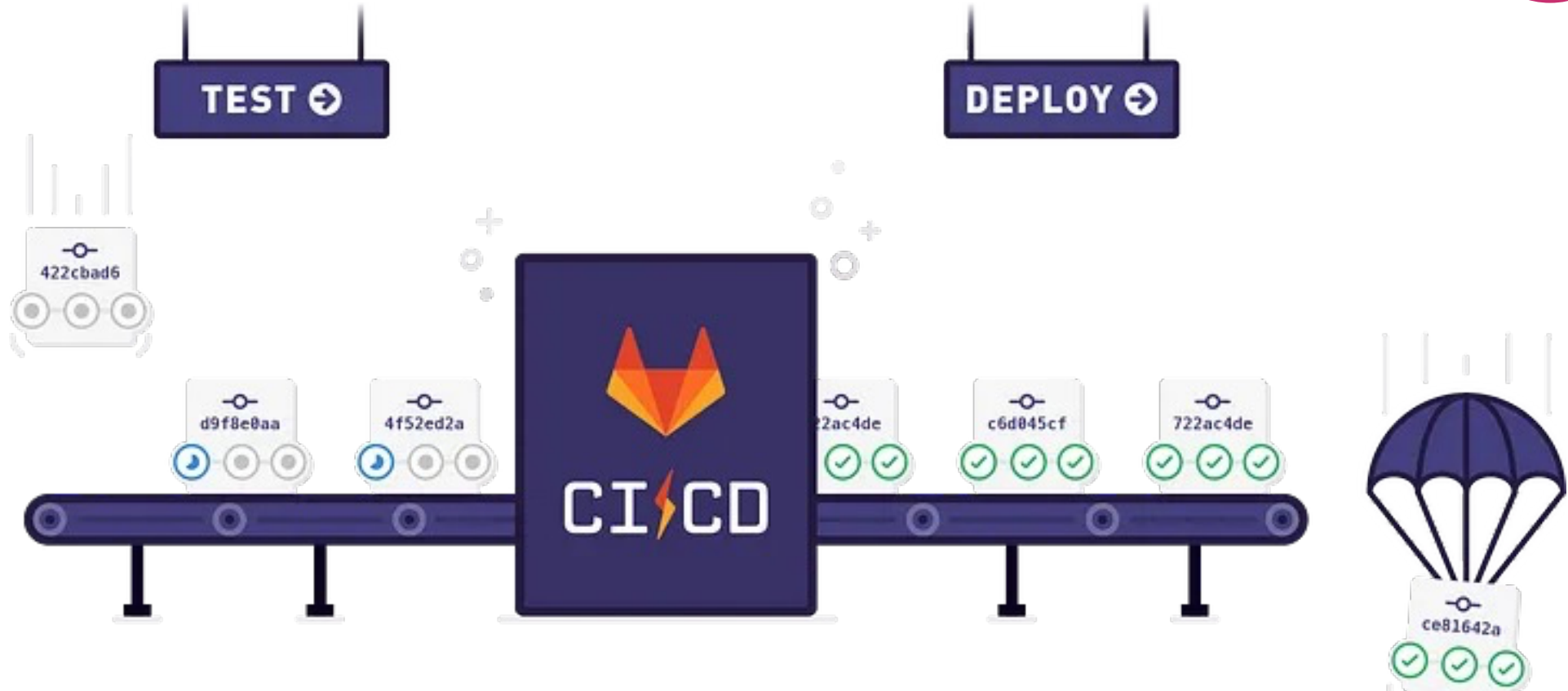
Git CI/CD

What is CI/CD?



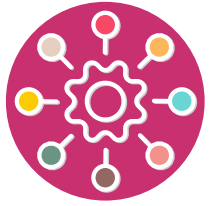
- Continuous Integration (CI) is the practice of frequently integrating code changes into a shared repository, which is then automatically built and tested.
- Continuous Delivery (CD) takes CI a step further by automatically deploying code changes to production-like environments for further testing and validation.
- Automated testing is a critical component of CI/CD, as it helps catch bugs and other issues early in the development process.
- Popular tools are GitLab CI/CD, Github Actions, Jenkins, CircleCI, and Travis CI.

What is CI/CD?

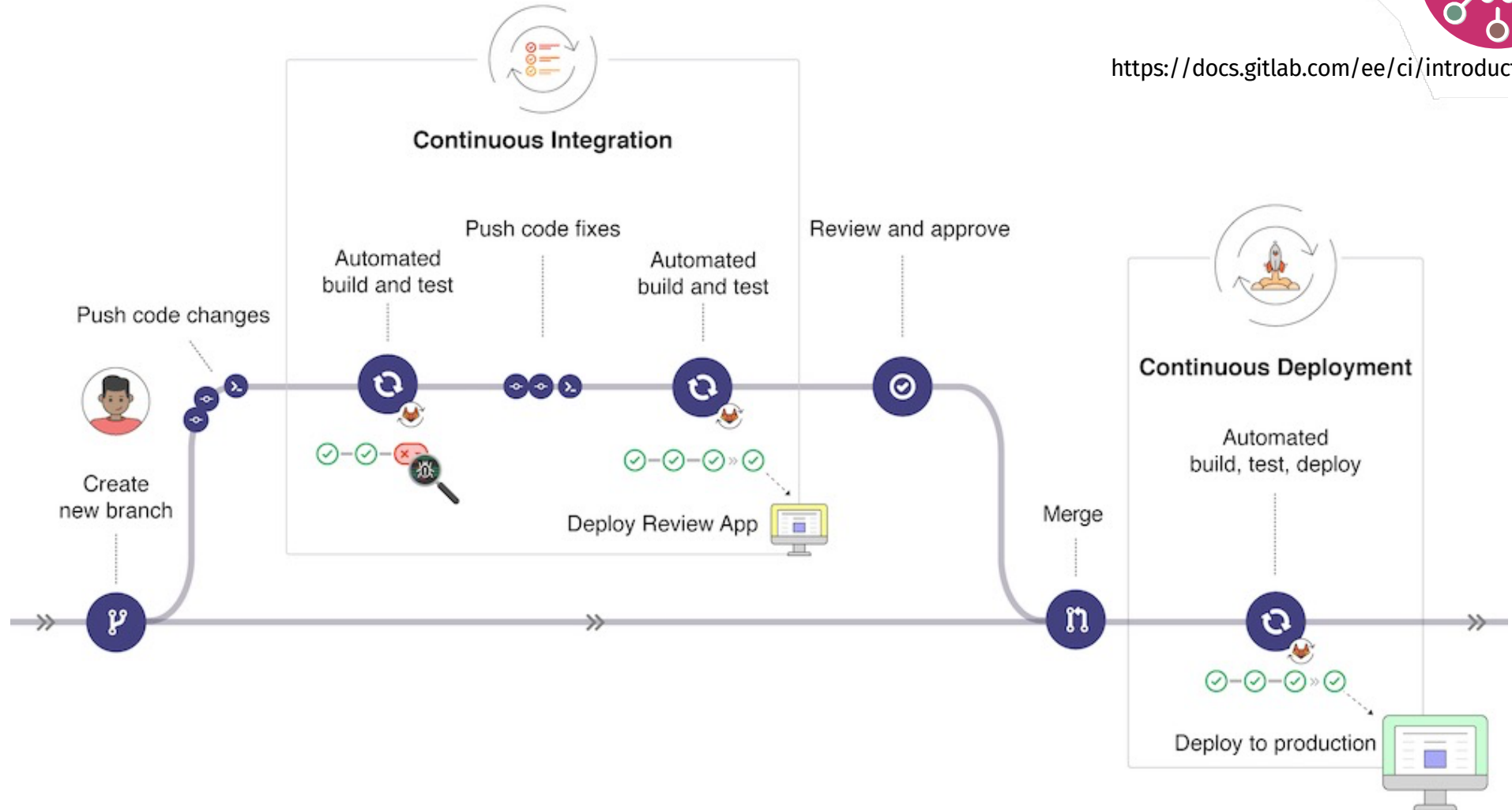


<https://about.gitlab.com/features/continuous-integration/>

Gitlab Workflow



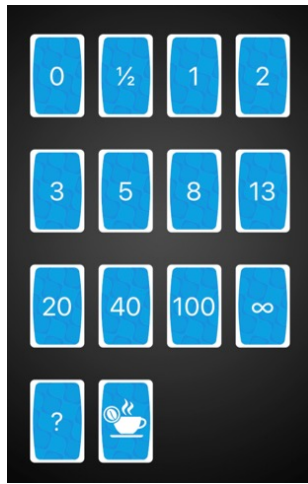
<https://docs.gitlab.com/ee/ci/introduction/>



Task Estimation

Estimation 4

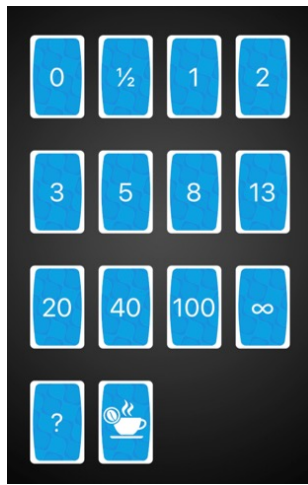
Create a Star Wars
Star Destroyer (4784pcs)
X Point(s)



Task Estimation

Estimation 5

Create a Star Wars
Star Destroyer (37pcs)
X Point(s)





SYSTEMS DESIGN / INTRODUCTION TO GIT - PART A



Introduction to git - Part A

Installation & Setup



Contents

1 Goal	1
2 Installation	2
3 Markdown	5
4 Outro	7
A GIT commands	8
B Most used Git commands	9

SYSTEMS DESIGN / INTRODUCTION TO GIT - PART B



Introduction to git - Part B



Contents

1 Goals	2
2 Outils	2
3 Basis Operationen	3
4 Branch and Merge	11
5 Gitgraph	15
6 Gitflow	16
7 Extras	18